



## Programmation 1 – Épreuve de seconde chance

*Durée : 3 heures.*

*Les documents manuscrits et dactylographiés sont autorisés. Les exercices sont indépendants. La clarté et la précision de la rédaction (en particulier les commentaires du code) seront un facteur important de l'évaluation. Le barème est donné à titre indicatif.*

### Exercice 1

On considère la gestion des candidatures pour les formations d'une université.

```
1  /**
2   * Classe représentant les candidats
3   */
4  public class Applicant {
5      public int id;
6      public String name
7      public String birthDate;
8      ...
9  }
```

```
1  /**
2   * Classe gérant les candidatures à une formation. Les clés sont
3   * des identifiants des candidats, et les valeurs les candidats
4   * pour cette formation.
5   *
6   * Par exemple :
7   *     1001 -> (1001, "Barnabé Vlk", "10/10/99")
8   *     3009 -> (3009, "Lucrèce Lewkeeratiyutkul", "03/06/99")
9   */
10 public class Applications extends Table<Integer, Applicant>
```

```
1  /**
2   * Classe gérant les candidatures à des formations. Les clés sont
3   * des noms de formation, et les valeurs les ensembles des tables
4   * de candidats (non vides) pour cette formation.
5   *
6   * Par exemple :
```

```
7 * "Licence 3 Info" ->
8 *      [ 1001 -> (1001, "Barnabé Vlk", "10/10/99"),
9 *      3009 -> (3009, "Lucrèce Lewkeeratiyutkul", "03/06/99") ]
10 * "Licence 3 Miage" ->
11 *      [ 1001 -> (1001, "Barnabé Vlk", "10/10/96"),
12 *      5542 -> (5542, "Ildéphonse Dupont", "25/01/95") ]
13 */
14 public class StudentApplications extends Table<String, Applications>
```

Les réponses seront fournies à l'aide des méthodes des classes *Table* et *Set* vues en TD. Écrire les méthodes d'instance suivantes de la classe *StudentApplications*:

### Question 1.1 (1 pt)

```
1 /**
2  * @param program      nom de la formation
3  * @param applicant    candidat
4  * @return true si applicant postule à la formation program,
5  *         false sinon
6  */
7 public boolean hasApplied(String program, Applicant applicant)
```

### Question 1.2 (2 pt)

```
1 /**
2  * @param applicant    candidat
3  * @return nombre de candidatures de applicant dans toutes les
4  *         formations
5  */
6 public int numberOfApplications(Applicant applicant)
```

### Question 1.3 (3 pt)

```
1 /**
2  * @return nombre d'étudiants qui ont postulé dans au moins
3  *         une formation.
4  */
5 public int applicantNumber()
```

**Question 1.4 (4 pt)**

```
1  /**
2   * Mettre à jour les formations dans lesquelles postule applicant.
3   * @post applicant est candidat exclusivement pour les formations
4   *       appartenant à l'ensemble programs.
5   * @param applicant      candidat
6   * @param programs      Ensemble de formations où il postule
7   */
8  public void modifyPrograms(Applicant applicant, Set<String> programs)
```

**Exercice 2**

On s'intéresse à des arbres binaires à valeurs entières positives. Ces arbres sont organisés pour une recherche symétrique (comme un arbre binaire de recherche) relativement à ces valeurs entières.

Écrire les méthodes statiques suivantes :

**Question 2.1 (1 pt)**

```
1  /**
2   * Afficher dans l'ordre croissant les nombres présents dans
3   * l'arbre tree.
4   *
5   * @param      tree      arbre binaire
6   */
7  public static void printValues(BinaryTree<Integer> tree)
```

**Question 2.2 (2 pt)**

```
1  /**
2   * @param      tree      arbre binaire
3   * @param      depth     profondeur minimale
4   * @return      nombre de nœuds de this se trouvant à
5   *              une profondeur supérieure ou égale à depth
6   */
7  public static boolean countValues(BinaryTree<Integer> tree, int depth)
```

**Question 2.3 (3pts)**

```
1  /**
2   * @param tree1          premier arbre binaire
3   * @param tree2          second arbre binaire
4   * @return true si les arbres tree1 et tree2 ont la même structure
5   *          d'arbre. C'est-à-dire que chaque nœud a le même
```

[illegible]

### Question 2.4 (4 pt)

On considère les objets suivants :

```
1 public class MyPair {
2     public int value, balance;
3     public MyPair () { value = balance = 0; }
4     public MyPair (int value, int balance) {
5         this.value = value;
6         this.balance = balance;
7     }
8     ...
9 }
```

```

1  /**
2   * @pre      !tree.isEmpty()
3   * @return   l'arbre binaire ayant la même structure que tree, mais
4   *           dans lequel les valeurs des nœuds sont des doublets
5   *           (value, balance) où value est la même que celle de ce nœud
6   *           dans tree et balance est égal à -1 si le nombre de
7   *           nœuds dans le sous-arbre gauche est supérieur au
8   *           nombre de nœuds dans le sous-arbres droit, est égal à 0
9   *           si les nombres de nœuds dans les 2 sous-arbres sont les
10  *           mêmes, et 1 sinon.
11  */
12 public static BinaryTree<MyPair> balanceTree(BinaryTree<Integer> tree)

```

Exemple :

